# Summarization Service Test Documentation

## Purpose

This document explains how to run and understand the test suite for the summarization microservice: a FastAPI service that either summarizes submitted text or refuses it when it violates a safety policy (harmful, illicit, or financial-advice content).

## Local setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install flake8 pylint radon mutmut
```

## Starting the service

```
python -m uvicorn app.main:app --reload
```

The service listens on `http://127.0.0.1:8000` by default.

## Running tests

With the service running in one terminal, run the HTTP-level suite from another:

```
pytest tests/test_summarize.py -v
```

`tests/test_app_unit.py` and `tests/test_cli.py` import application code directly and don't need the service running:

```
pytest tests/test_app_unit.py tests/test_cli.py -v
```

Or run everything at once (the live-service tests will fail/error if uvicorn isn't running):

```
pytest -v
```

# Test cases

Each entry below names the exact `project_inputs/` fixture file the test reads (where one is used) and the test function that reads it, so the fixture-to-test mapping is traceable without reading the source.

## Health endpoint

- **Purpose:** confirm the service process is up and responding.
- **Fixture:** none — no input file needed for a liveness check.
- **Test:** `test_health_endpoint` in `tests/test_summarize.py`.
- **Expected result:** `GET /health` returns `200` with `{"status": "ok"}`.
- **Policy connection:** none; this is a liveness check.

## Valid long-form summarization

- **Purpose:** confirm a normal, policy-clean document is summarized rather than refused.
- **Fixture:** `project_inputs/Long_text.txt`, read in full as the request body's `text` field.
- **Test:** `test_summarize_valid_long_text` in `tests/test_summarize.py`.
- **Expected result:** `POST /summarize` returns `200` with a non-empty `summary` string shorter than the input, and `refused: false`.

## Response schema

- **Purpose:** lock the response contract so client code can rely on it.
- **Fixture:** none — uses an inline literal string, since the schema check doesn't depend on any specific fixture content.
- **Test:** `test_response_shape_schema_fields` in `tests/test_summarize.py`.
- **Expected result:** the response always contains `summary` (str), `word_count` (int, equal to `len(summary.split())`), `source_id` (str, echoing the request's `source_id`), `refused` (bool), `policy_code` (str or null), and `message` (str or null).

## Harmful / illicit / financial-advice refusal

- **Purpose:** confirm requests seeking harmful, illegal, or guaranteed-return-style financial content are refused rather than summarized.
- **Fixture:** `project_inputs/forbidden_examples.json` — every example under all three top-level categories (`harmful_content`, `illicit_instructions`, `financial_advice`) is used; none are sampled or skipped.
- **Test:** `test_forbidden_content_is_refused` in `tests/test_summarize.py`, parametrized once per fixture example via `load_forbidden_cases()`.

- **Policy codes:** `safety_harm`, `safety_illicit`, `safety_financial` respectively (each example's expected code comes from the fixture file itself, not a hard-coded value in the test).
- **Expected result:** `refused: true`, empty `summary`, `word_count: 0`, `message: "refused: policy_violation"`, and the original unsafe text is not echoed back in the response.

### Policies endpoint

- **Purpose:** confirm the list of supported policy codes is discoverable without reading the source.
- **Fixture:** none.
- **Test:** `test_policies_endpoint_returns_supported_policies` in `tests/test_summarize.py`.
- **Expected result:** `GET /policies` returns `200` with `{"policies": ["safety_harm", "safety_illicit", "safety_financial"]}`.

### Unit test suite (`tests/test_app_unit.py`)

- **Purpose:** exercise `app.main`'s functions and endpoints in-process (via FastAPI's `TestClient`) so mutation testing can actually detect code changes — see "Mutation testing" below for why this file exists separately from `tests/test_summarize.py`.
- **Fixture:** none from `project_inputs/`. Keyword/category cases are inline literals in the test file itself (one case per entry in `HARMFUL_KEYWORDS` / `ILLICIT_KEYWORDS` / `FINANCIAL_KEYWORDS`), so each individual keyword — not just one example per category — is covered.

### CLI test suite (`tests/test_cli.py`)

- **Purpose:** cover `scripts/run_tests.py`'s argument parsing and failure paths (missing dataset, unreachable service).
- **Fixture:** none from `project_inputs/` — tests write small throwaway JSON datasets to pytest's `tmp_path` fixture rather than reading the real `forbidden_examples.json`, since the goal is to test the CLI's own logic, not re-validate the dataset. The CLI itself defaults to `project_inputs/forbidden_examples.json` when run normally (see "Automation and CLI usage" below).

## TDD workflow

`app/main.py`'s `/policies` endpoint was built test-first as a worked example of red/green/refactor:

1. **Red** — a failing test (`test_policies_endpoint_returns_supported_policies`) was committed against a `404` (`test: add failing policies endpoint test`).
2. **Green** — the smallest possible endpoint implementation was added to make it pass (`feat: implement policies endpoint`).

3. **Refactor** — the policy code literals were centralized into constants used by both the endpoint and the detection logic, with the full suite staying green throughout (`refactor: centralize supported policy definitions`).

The commit history for this sequence is saved at `artifacts/tdd-commit-history.txt`.

## Complexity refactor

`project_inputs/review_comments.txt` asked for validation logic to be extracted out of the summarize handler and for repeated response construction to be centralized. `app/main.py` now has dedicated, docstringed helpers: `detect_policy_violation`, `resolve_source_id`, `build_refusal_response`, and `build_summary_response`, leaving the `/summarize` endpoint as a short coordinator.

Radon complexity reports before and after the refactor are saved under `artifacts/complexity/` (`radon-before.txt` / `radon-after.txt` / `comparison.txt`). The endpoint's core logic dropped from cyclomatic complexity B(7) to A(4)/A(2), and the file's average dropped from A(2.75) to A(2.0), with the full test suite green before and after.

## Mutation testing

Mutation testing (via `mutmut`) mutates `app/main.py` (flips comparisons, changes constants, etc.) and checks whether the test suite notices. It needs tests that import the application in-process — the live-service tests in `tests/test_summarize.py` can't detect mutations because they talk to an already-running, unmutated uvicorn process. `tests/test_app_unit.py` exists for this: it imports `app.main` directly via FastAPI's `TestClient`.

```
mutmut run
mutmut results
```

Initial run: 39/75 mutants killed (score 0.52). After strengthening the unit tests (see `artifacts/mutation/summary.md` for the full breakdown — tautological constant comparisons, missing per-keyword coverage, and untested boundary conditions were the main gaps), the final run killed 70/75 (score 0.93). The remaining 5 survivors are documented as non-actionable: 2 are a `mutmut`/pytest exit-code compatibility gap (verified by reproducing the mutant manually) and 3 are genuinely equivalent mutants (unused `Optional` field defaults).

## Test-data versioning

Every file under `project_inputs/` is hashed with SHA-256 and recorded in `project_inputs/manifest.json`, so it's possible to detect if a test input changed without anyone updating the corresponding test:

```
shasum -a 256 project_inputs/*
```

## Model-run evaluation

`artifacts/model-evaluation-report.md` compares the candidate model runs in `project_inputs/candidate_model_runs.json` on safety pass rate and utility score, and records which run was selected and why (safety is weighted as the primary criterion for this service, since a false negative here means unsafe content gets summarized instead of refused).

## Automation and CLI usage

`scripts/run_validation.sh` is the one-shot CI entry point: it starts uvicorn, waits for `/health`, runs pytest/flake8/pylint/radon against the running service, archives the results to a tarball, and always stops the service on exit (even on failure).

```
./scripts/run_validation.sh
```

`scripts/run_tests.py` is a standalone CLI for replaying a forbidden-examples-style dataset against any running instance of the service (useful for smoke-testing a deployed environment, not just localhost):

```
python scripts/run_tests.py \
  --base-url http://127.0.0.1:8000 \
  --dataset project_inputs/forbidden_examples.json \
  --timeout 5 \
  --artifact-dir artifacts/cli-run
```

It exits non-zero if the dataset path is missing/invalid, the service is unreachable, or any case fails, and writes a `results.json` with a per-case pass/fail breakdown. `tests/test_cli.py` covers its argument parsing and both failure paths.

## CI execution

A CI job should, in order: install dependencies, run `./scripts/run_validation.sh` (which covers tests + lint + complexity against a live instance), then optionally run `mutmut run` and `python scripts/run_tests.py` against a deployed environment as a post-deploy smoke check. Non-zero exit from any step should fail the job.